



Prova Finale - Progetto di Reti Logiche

Anno scolastico 2024/2025

Liao Emiliano

Codice Persona: 10889441

Contents

1	Introduzione	3
1.1	Specifica	3
1.2	Funzionamento	3
2	Architettura	4
2.1	Elementi di memoria	4
2.2	FSM	5
2.3	Processi	5
2.3.1	StateProcess	5
2.3.2	ControllProcess	5
2.3.3	raProcess_seq	6
2.3.4	raProcessCom	6
2.3.5	waProcess	6
2.3.6	hc_process_seq	6
2.3.7	hc_process_com	6
2.3.8	memMux_process	6
2.3.9	loadHeaderProcess	6
2.3.10	activeCoeffProcess	6
2.3.11	count_process_seq	7
2.3.12	count_process_com	7
2.3.13	bufferProcess	7
2.3.14	sumProcess	7
2.3.15	res_norm_process_seq	7
2.3.16	res_norm_process_com	7
2.3.17	k_process_com	7
3	Risultati sperimentali	8
3.1	Sintesi	8
3.2	Simulazioni	8
3.2.1	Test 1	9
3.2.2	Test 2	9
3.2.3	Test 3	10
4	Conclusioni	10

1 Introduzione

1.1 Specifica

Il modulo da implementare ha i seguenti ingressi:

- **segnale i_clk:** 1 bit, rappresenta il segnale di clock
- **segnale i_rst:** 1 bit, rappresenta il segnale di reset
- **segnale i_start:** 1 bit, rappresenta il segnale di start
- **segnale i_add:** 16 bit, indica l'indirizzo di memoria iniziale
- **segnale i_mem_data:** rappresenta il valore letto da memoria

e le seguenti uscite:

- **segnale o_mem_en:** 1 bit, segnale che abilita la memoria
- **segnale o_mem_we:** 1 bit, segnale che abilita la scrittura in memoria
- **segnale o_done:** 1 bit, segnale di terminazione del processo
- **segnale o_mem_addr:** 16 bit, indirizzo di lettura/scrittura in memoria
- **segnale o_mem_data:** 16 bit, dato in uscita

Il progetto deve funzionare con un periodo di clock di almeno 20ns e non deve contenere latch se non esplicitamente specificati.

1.2 Funzionamento

Il componente legge da memoria una header lunga 17 byte contenente le informazioni riguardanti i valori di K, S, e coefficienti usati per il calcolo. In base al bit meno significativo di S vengono selezionati i coefficienti da utilizzare e il coefficiente di normalizzazione. Dopo la lettura del header, viene letta byte a byte la sequenza delle R word a cui viene applicato un'operazione di filtro e normalizzazione; dopodiché il risultato viene riscritto in memoria a partire dall'indirizzo $i_add + 17 + K$.

Il modulo rimarrà in attesa finché il segnale i_rst non diventi 0 e il segnale di start non diventi 1. Terminata la sequenza delle R word, porta a 1 il segnale o_done. Un nuovo segnale di start non può essere dato fin tanto che o_done è a 1.

2 Architettura

2.1 Elementi di memoria

Il componente progettato utilizza i seguenti registri di memoria:

- Registro K1: registro a 8 bit usato per memorizzare i bit più significativi di K.
- Registro K_start: registro a 16 bit, memorizza il valore iniziale di K.
- Registro K: registro a 16 bit, contiene il valore di K che viene decrementato per ogni word R letto, filtrato e scritto in memoria.
- S: registro a 8 bit, memorizza il valore di S.
- coeff_3: array di 7 elementi, ciascuna da 8 bit, memorizza i coefficienti di filtro di ordine 3. coeff_5: array di 7 elementi, ciascuna da 8 bit, memorizza i coefficienti di filtro di ordine 5.
- active_coeff: array di 7 elementi, ciascuna da 8 bit, memorizza i coefficienti selezionati con l'ultimo bit di S.
- buffer_R: array di 7 elementi, ciascuna da 8 bit, memorizza le word R usate nel filtraggio.
- sum: registro a 32 bit, memorizza il risultato della sommatoria del filtraggio.
- res_norm: registro a 32 bit, memorizza la somma normalizzata.
- final_res: registro a 8 bit, memorizza il valore normalizzato dopo resize su 8 bit, è il valore che verrà scritto in memoria.
- hc_count: registro a 5 bit, memorizza un contatore usato in fase di lettura della header.
- buffer_count: registro a 16 bit, memorizza un contatore usato in fase di caricamento dei primi valori nel buffer. Se si usa il filtro di ordine 3, assume valore -2, altrimenti valore -3.
- read_addr: registro a 16 bit, memorizza l'indirizzo di lettura.
- ra_lim: registro a 16 bit, memorizza l'indirizzo limite per la lettura.
- write_addr: registro a 16 bit, memorizza l'indirizzo di scrittura.

2.2 FSM

I componenti sono controllati da una FSM centrale, rappresentata in figura 1.

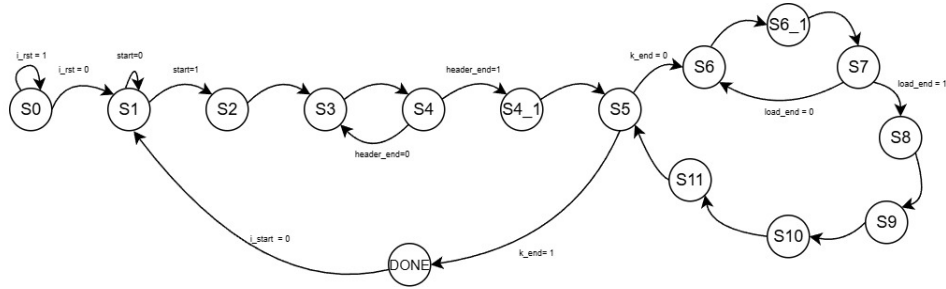


Figure 1: Finite State Machine del componente

La FSM è composta da 14 stati:

S0: stato di reset, il sistema è in attesa che `i_rst` diventi 0.

S1: stato di attesa del segnale di start.

S2: stato di inizializzazione, il sistema inizializza il buffer R , *il registro read_addr il header_count*.

S3: stato di abilitazione della memoria in lettura per la lettura del header.

S4: stato di scrittura del dato letto e incremento dell'indirizzo di lettura e del header_count.

S4_1: stato di inizializzazione del count usato in fase di calcolo e del registro *write_addr*.

S5: stato di check, verifica che `o_done` sia uguale a 0 per continuare l'esecuzione.

S6: stato di abilitazione della memoria in lettura per la lettura delle word R .

S6_1: stato di scrittura nel buffer il dato letto.

S7: stato di incremento dell'indirizzo di lettura e del count.

S8: stato di calcolo della somma.

S9: stato di calcolo del risultato normalizzato, e del risultato dopo resize.

S10: stato di abilitazione della memoria per la scrittura del risultato.

S11: stato di scrittura in memoria, incremento dell'indirizzo di scrittura e decremento di K .

DONE: stato di terminazione.

2.3 Processi

Di seguito la descrizione dei processi che si occupano della gestione delle parti interne del componente.

2.3.1 StateProcess

Processo sequenziale che si occupa della gestione dei cambi di stato della FSM. Ad ogni ciclo di clock, viene cambiato lo stato memorizzato in **curr_state** in base allo stato precedente e ai segnali attivi in quel ciclo di clock.

2.3.2 ControllProcess

Processo combinatorio, ad ogni cambio di stato, reimposta i valori di default dei segnali interni (valore 0), e attiva i segnali necessari in base allo stato corrente.

2.3.3 raProcess_seq

Processo sequenziale, si occupa della gestione del registro **read_addr**, in base ai segnali attivi, esegue l'operazione di inizializzazione a 0, assegnazione o incremento del valore in **read_addr**.

2.3.4 raProcessCom

Processo combinatorio che si attiva ogni volta che **read_addr** cambia valore. Se il valore in **read_addr** supera il limite salvato in **ra_lim**, imposta a 1 il valore del segnale **ra_end**, impedendo così la lettura ad indirizzi di memoria oltre il limite.

2.3.5 waProcess

Processo sequenziale, si occupa della gestione del registro **write_addr**, in base ai segnali attivi, esegue l'operazione di inizializzazione a 0, assegnazione o incremento del valore in **write_addr**.

2.3.6 hc_process_seq

Processo sequenziale, si occupa della gestione del registro **header_count**, in base ai segnali attivi, esegue l'operazione di inizializzazione a 0, o incremento del valore in **header_count**.

2.3.7 hc_process_com

Processo combinatorio che si attiva ogni volta che **header_count** cambia valore. Se **header_count** raggiunge il valore 16, imposta a 1 il segnale **header_end**, che permette alla FSM di passare dallo stato **S4** allo stato **S4_1**, in quanto la lettura dei byte della header è conclusa.

2.3.8 memMux_process

Processo combinatorio con funzione di multiplexer. Si occupa della selezione dell'indirizzo da assegnare a **o_mem_addr**, se il segnale **add_mux_sel** = 1, assegna l'indirizzo presente in **write_addr**, altrimenti assegna l'indirizzo presente in **read_addr**.

2.3.9 loadHeaderProcess

Processo sequenziale che si occupa del caricamento dei byte della header nei appositi registri. In base al valore del **header_count**, scrive i valori nei registri **K1**, **K**, **S**, **coeff_3** e **coeff_5**. Dato che il segnale assegna valore al registro **K**, si occupa anche dell'operazione di decremento di **K**. Questo per evitare errori di multiple drivers.

2.3.10 activeCoeffProcess

Processo sequenziale che si occupa di selezionare i coefficienti di filtro di ordine 3 o 5 da utilizzare per l'intera sequenza di esecuzione. Se il bit menosignificativo di **S** è 1 copia dall'array **coeff_5**, altrimenti copia dall'array **coeff_3**.

2.3.11 count_process_seq

Processo sequenziale che si occupa della gestione del registro **buffer_count**. Assegna il valore iniziale del contatore in base al bit meno significativo del registro **S**, e successivamente si occupa di incrementare il valore del contatore.

2.3.12 count_process_com

Processo combinatorio che si occupa di verificare la terminazione del caricamento iniziale del **buffer_R**. Se il valore di **buffer_R** è maggiore o uguale a 0, imposta a 1 il valore del segnale **load_end**, permettendo quindi al FSM di passare dallo stato **S8** per procedere al calcolo del filtraggio.

2.3.13 bufferProcess

Processo sequenziale che si occupa della gestione del **buffer_R**. Il **buffer_R** è un array implementato con logica FIFO, quando bisogna aggiungere un nuovo elemento nel buffer, prima si effettua uno spostamento di tutti gli elementi di una posizione in avanti. Nel calcolo della somma del filtro, se è di ordine 3, vengono considerati solo gli elementi da indice 2 a 6, mentre se di ordine 5 vengono considerati tutti gli elementi.

2.3.14 sumProcess

Processo sequenziale che effettua il calcolo della somma del filtro. La somma è calcolata su 32 bit per evitare casi di overflow.

2.3.15 res_norm_process_seq

Processo sequenziale che si occupa della normalizzazione della somma. In base al bit meno significativo di **S**, viene fatta la normalizzazione con coefficiente 1/12 oppure 1/60. La normalizzazione è fatta utilizzando il comando **shift_right** ed è rappresentata ancora su 32 bit. Se la somma è negativa, viene applicato un'approssimazione di +1.

2.3.16 res_norm_process_com

Processo combinatorio che si occupa di effettuare il controllo del valore di **res_norm**. Se **res_norm** supera i valori compresi tra [-128;127], viene saturato con i valori limite. In ogni caso, viene fatto un resize al valore di **res_norm** su 8 bit.

2.3.17 k_process_com

Processo combinatorio che verifica il valore di **K**. Quando **K** diventa 0, imposta il livello del segnale **k_end** a 1 per terminare la sequenza di esecuzione.

3 Risultati sperimentali

3.1 Sintesi

Il componente progettato è sintetizzabile ed è simulabile correttamente anche post sintesi. Tutti gli elementi di memoria sono implementati usando flip-flop, non sono presenti elementi di latch come riportato in figura 2, risultato del comando **Report_utilization**; e rispetta il constrain di periodo di clock, come riportato in figura 3, si ha uno **Slack time** di 8.803ns.

Site Type	Used	Fixed	Prohibited	Available	Util%
Slice LUTs*	1276	0	0	134600	0.95
LUT as Logic	1276	0	0	134600	0.95
LUT as Memory	0	0	0	46200	0.00
Slice Registers	398	0	0	269200	0.15
Register as Flip Flop	398	0	0	269200	0.15
Register as Latch	0	0	0	269200	0.00
F7 Muxes	0	0	0	67300	0.00
F8 Muxes	0	0	0	33650	0.00

Figure 2: Report di utilization

```
Timing Report

Slack (MET) : 8.803ns (required time - arrival time)
Source:      active_coeff_reg[5][3]/C
              (rising edge-triggered cell FDCE clocked by clock {rise@0.000ns fall@10.000ns period=20.000ns})
Destination: sum_reg[29]/D
              (rising edge-triggered cell FDCE clocked by clock {rise@0.000ns fall@10.000ns period=20.000ns})
Path Group:  clock
Path Type:   Setup (Max at Slow Process Corner)
Requirement: 20.000ns (clock rise@20.000ns - clock rise@0.000ns)
Data Path Delay: 11.079ns (logic 5.239ns (47.288%) route 5.840ns (52.712%))
Logic Levels: 15 (CARRY4=8 LUT3=2 LUT5=1 LUT6=4)
```

Figure 3: Report di timing

3.2 Simulazioni

Tutte i test bench sono stati eseguiti sia in pre sintesi che post sintesi; il componente ha concluso tutte le simulazioni fatte con esito positivo in entrambi i casi. Di seguito sono stati riportati alcuni risultati di testing.

3.2.1 Test 1

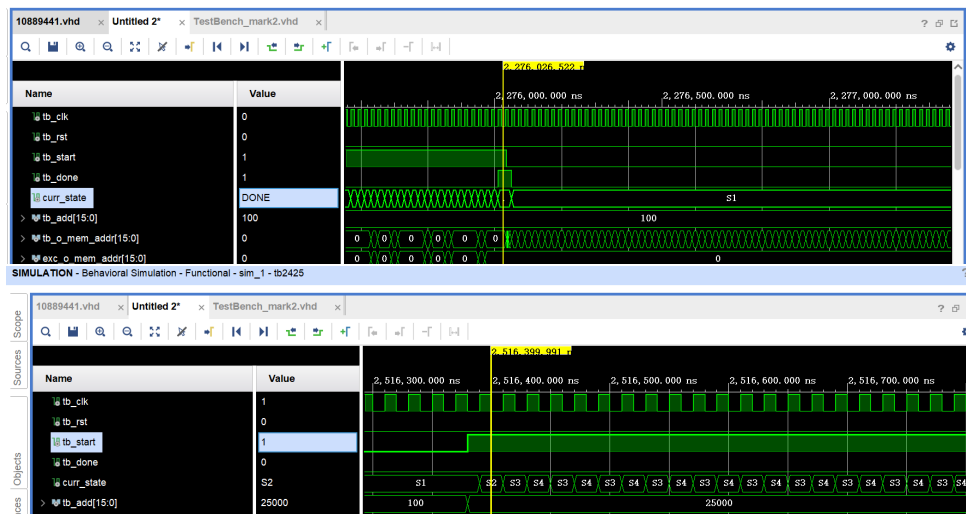


Figure 4: Test esecuzioni in sequenza

Il test bench prevede due sequenze di input e output attesi, senza un segnale di reset intermedio, le immagini riportate mostrano il corretto funzionamento del componente in questo caso di test. Dopo aver terminato l'esecuzione della prima sequenza di dati, viene portato a 1 il segnale `o_done`, quindi la FSM torna allo stato `S1` e attende che il segnale di `i_start` diventi 1 per proseguire con l'esecuzione della seconda sequenza di dati.

3.2.2 Test 2

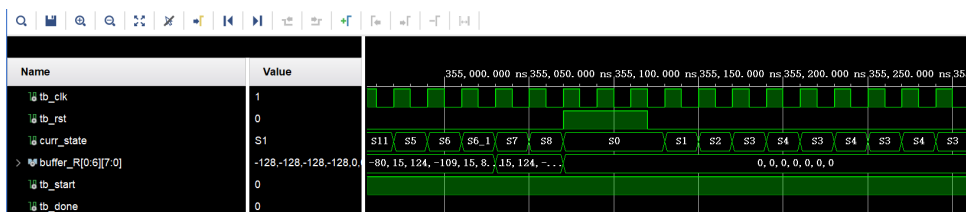


Figure 5: Test con reset intermedio

Questo test mostra come anche ricevendo un segnale di reset dopo che è iniziata la fase di esecuzione, il componente è in grado comunque di ricominciare dallo stato `S0` e di terminare correttamente l'esecuzione del test bench.

3.2.3 Test 3

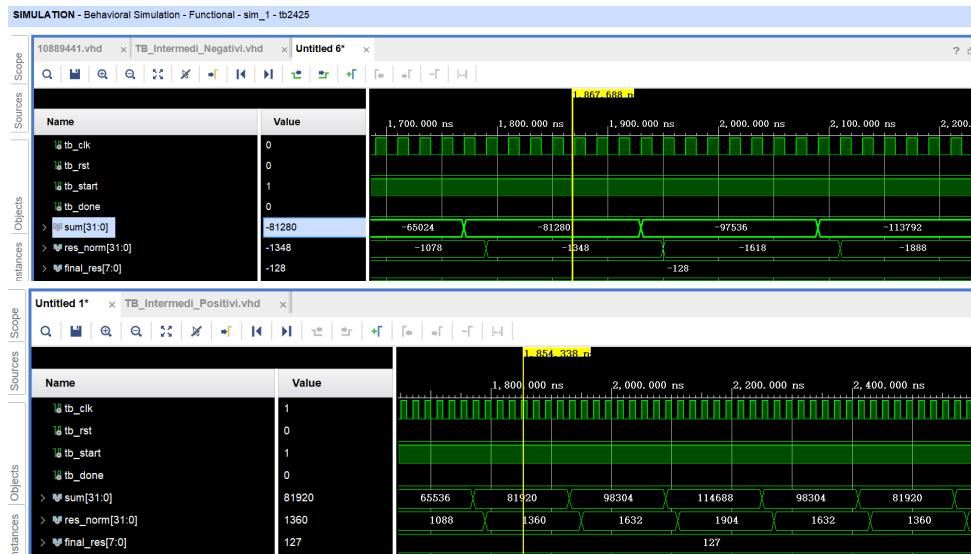


Figure 6: Test saturazione

Il test bench da in input valori che dopo il filtraggio risultano valori non rappresentabili nel range $[-128;127]$, la figura mostra come il valore viene correttamente saturato al limite, funzionamento atteso dalle specifiche.

4 Conclusioni

Il componente progettato è stato sviluppato secondo le specifiche richieste, sono rispettati i vincoli di clock e non contiene nessun elemento che utilizzi latch asincroni. La suddivisione del codice in più processi anche per un singolo elemento è stata fatta in modo da facilitarne la lettura e le successive operazioni di debug.